In [31]:
```python
import sys
import keyword
import operator
from datetime import datetime
import os
```

keyword

Keywords are the reserved words in Python and can't be used as an identifier

In [5]:
```python
print(keyword.kwlist)
```

```
['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await', 'break', 'class',
'continue', 'def', 'del', 'elif', 'else', 'except', 'finally', 'for', 'from', 'globa
l', 'if', 'import', 'in', 'is', 'lambda', 'nonlocal', 'not', 'or', 'pass', 'raise',
'return', 'try', 'while', 'with', 'yield']
```

In [8]:
```python
len(keyword.kwlist)
```

Out[8]:
```
35
```

Identifiers

An identifier is a name given to entities like class, functions, variables, etc. It helps to differentiate one entity from another.

In [6]:
```python
1var = 10 # Identifier can't start with a digit
```

```
  Cell In[6], line 1
    1var = 10 # Identifier can't start with a digit
    ^
SyntaxError: invalid decimal literal
```

In [7]:
```python
val2@ = 35 # Identifier can't use special symbols
```

```
  Cell In[7], line 1
    val2@ = 35 # Identifier can't use special symbols
          ^
SyntaxError: invalid syntax
```

In [8]:
```python
import = 125 # Keywords can't be used as identifiers
```

```
  Cell In[8], line 1
    import = 125 # Keywords can't be used as identifiers
           ^
SyntaxError: invalid syntax
```

In [9]:
```python
"""Correct way of defining an identifier
(Identifiers can be a combination of letters in lowercase (a to z) or uppercase """
```

```
val2 = 10
```

In [10]:
```
val_ = 99
```

Comments in Python

Comments can be used to explain the code for more readabilty.

In [13]:
```
# Single line comment
val1 = 10
```

In [14]:
```
# Multiple # line
# comment
val1 = 10
```

In [15]:
```
'''
Multiple line
comment '''
val1 = 10
```

In [16]:
```
"""
Multiple line
comment """
val1 = 10
```

Statements

Instructions that a Python interpreter can execute

In [17]:
```
p = 20 #Creates an integer object with value 20 and assigns the variable p to p
q = 20 # Create new reference q which will point to value 20. p & q will be poi
r = q # variable r will also point to the same location where p & q are pointin
p , type(p), hex(id(p)) # Variable P is pointing to memory location '0x7fff6d71a
```

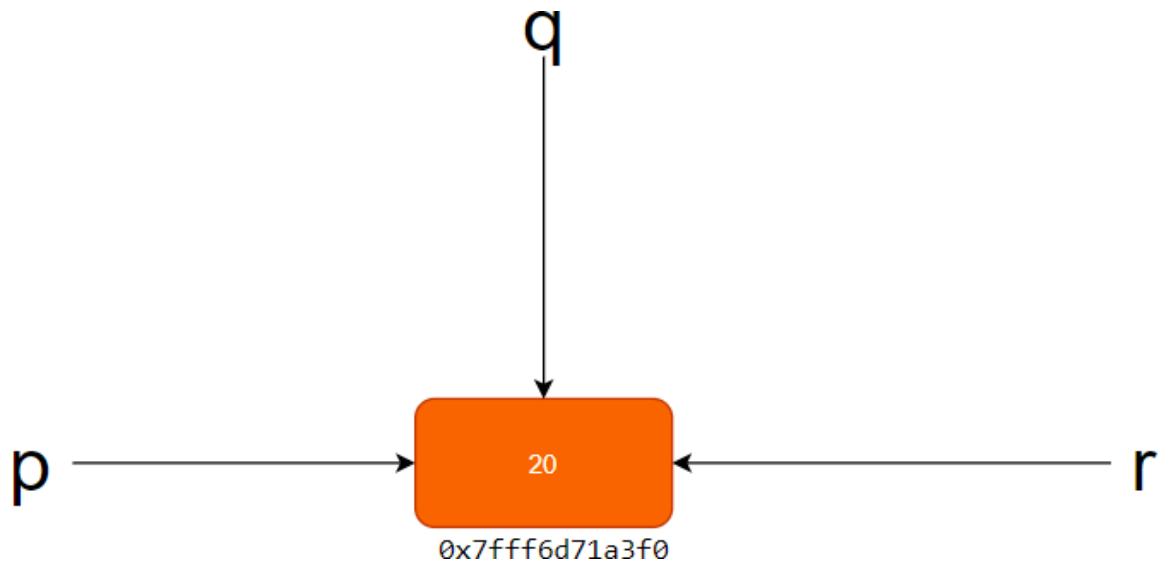
Out[17]:
```
(20, int, '0x7ffecc3fa608')
```

In [18]:
```
q , type(q), hex(id(q))
```

Out[18]:
```
(20, int, '0x7ffecc3fa608')
```

In [19]:
```
r , type(r), hex(id(r))
```

Out[19]:
```
(20, int, '0x7ffecc3fa608')
```



In [20]:
```
p = 20
p = p + 10 # Variable Overwriting
p
```

Out[20]: 30

Variable Assigment

In [22]:
```
intvar = 10 # Integer variable
floatvar = 2.57 # Float Variable
strvar = "Python Language" # String variable
print(intvar)
print(floatvar)
print(strvar)
```

```
10
2.57
Python Language
```

Multiple Assignments

In [23]:
```
intvar , floatvar , strvar = 10,2.57,"Python Language" # Using commas to separat
print(intvar)
print(floatvar)
print(strvar)
```

```
10
2.57
Python Language
```

In [24]:
```
p1 = p2 = p3 = p4 = 44 # All variables pointing to same value
print(p1,p2,p3,p4)
```

```
44 44 44 44
```

Data Types

Numeric

In [38]:
```
val1 = 10 # Integer data type
print(val1)
print(type(val1)) # type of object
print(sys.getsizeof(val1)) # size of integer object in bytes
print(val1, " is Integer?", isinstance(val1, int)) # val1 is an instance of int
```

```
10
<class 'int'>
28
10  is Integer? True
```

In [33]:
```
val2 = 92.78 # Float data type
print(val2)
print(type(val2)) # type of object
print(sys.getsizeof(val2)) # size of float object in bytes
print(val2, " is float?", isinstance(val2, float)) # Val2 is an instance of floa
```

```
92.78
<class 'float'>
24
92.78  is float? True
```

In [34]:
```
val3 = 25 + 10j # Complex data type
print(val3)
print(type(val3)) # type of object
print(sys.getsizeof(val3)) # size of float object in bytes
print(val3, " is complex?", isinstance(val3, complex)) # val3 is an instance of
```

```
(25+10j)
<class 'complex'>
32
(25+10j)  is complex? True
```

In [43]:
```
sys.getsizeof(int()) # size of integer object in bytes
```

Out[43]: 28

In [40]:
```
sys.getsizeof(float()) # size of float object in bytes
```

Out[40]: 24

In [41]:
```
sys.getsizeof(complex()) # size of complex object in bytes
```

Out[41]: 32

Boolean

Boolean data type can have only two possible values true or false

In [44]:
```
bool1 = True
```

In [45]:
```
bool2 = False
```

In [46]:
```
print(type(bool1))
```

```
<class 'bool'>
```

In [67]:
```
print(type(bool2))
```

```
<class 'bool'>
```

In [61]:
```
isinstance(bool1, bool)
```

Out[61]:
```
True
```

In [64]:
```
isinstance(bool1, int)
```

Out[64]:
```
True
```

In [65]:
```
isinstance(bool1, float)
```

Out[65]:
```
False
```

In [66]:
```
isinstance(bool1, complex)
```

Out[66]:
```
False
```

In [68]:
```
bool(0)
```

Out[68]:
```
False
```

In [69]:
```
bool(1)
```

Out[69]:
```
True
```

In [70]:
```
bool(None)
```

Out[70]:
```
False
```

In [71]:
```
bool (False)
```

Out[71]:
```
False
```

Strings

String Creation

In [72]:
```
str1 = "HELLO PYTHON"
print(str1)
```

```
HELLO PYTHON
```

In [73]:
```
mystr = 'Hello World' # Define string using single quotes
print(mystr)
```

```
Hello World
```

In [74]:
```
mystr = "Hello World" # Define string using single quotes
print(mystr)
```

```
Hello World
```

In [75]:
```
mystr = '''Hello World''' # Define string using single quotes
print(mystr)
```

```
Hello World
```

In [78]:
```
mystr = """Hello
            World""" # Define string using triple quotes
print(mystr)
```

```
Hello
            World
```

In [79]:
```
mystr = ('Happy '
        'Monday '
         'Everyone')
print(mystr)
```

```
Happy Monday Everyone
```

In [80]:
```
mystr = ('Happy
        Monday
         Everyone')
print(mystr)
```

```
  Cell In[80], line 1
    mystr = ('Happy
             ^
SyntaxError: unterminated string literal (detected at line 1)
```

In [84]:
```
mystr = ("Happy
             Monday
             Everyone")
print(mystr)
```

```
  Cell In[84], line 1
    mystr = ("Happy
             ^
SyntaxError: unterminated string literal (detected at line 1)
```

In [85]:
```
mystr = ('''Happy
            Monday
            Everyone''')
print(mystr)
```

```
Happy
            Monday
            Everyone
```

In [87]:
```
mystr2 = 'Woohoo '
mystr2 = mystr2*5
mystr2
```

Out[87]:
```
'Woohoo Woohoo Woohoo Woohoo Woohoo '
```

In [88]:
```
len(mystr2) # Length of string
```

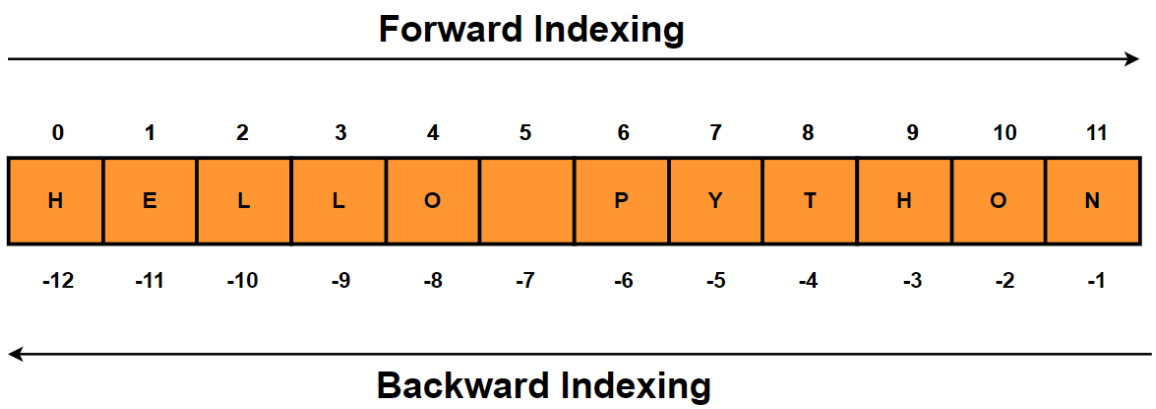
Out[88]:
```
35
```

String Indexing

In [ ]:

In [ ]:

In [ ]:



In [ ]:

In [89]:
```
str1
```

Out[89]:
```
'HELLO PYTHON'
```

In [90]:
```
str1[0] # First character in string "str1"
```

Out[90]:
```
'H'
```

In [91]:
```
len(str1)
```

Out[91]:
```
12
```

In [93]:
```
str1[len(str1)-1]
```

Out[93]: 'N'

In [94]:
```
str1[-1] # Last character in string
```

Out[94]: 'N'

In [95]:
```
str1[6] #Fetch 7th element of the string
```

Out[95]: 'P'

In [96]:
```
str1[5]
```

Out[96]: ' '

String Slicing

In [97]:
```
str1[0:5] # String slicing - Fetch all characters from 0 to 5 index location
```

Out[97]: 'HELLO'

In [98]:
```
str1[6:12] # String slicing - Retreive all characters between 6 - 12 index location
```

Out[98]: 'PYTHON'

In [109…
```
str1[-4:] # Retreive last four characters of the string
```

Out[109… 'THON'

In [110…
```
str1[-6:] # Retreive last six characters of the string
```

Out[110… 'PYTHON'

In [111…
```
str1[:4] # Retreive first four characters of the string
```

Out[111… 'HELL'

In [112…
```
str1[:6] # Retreive first six characters of the string
```

Out[112… 'HELLO '

Update & Delete String

In [113…
```
str1
```

Out[113… 'HELLO PYTHON'

In [116…
```
#Strings are immutable which means elements of a string cannot be changed once they
str1[0:5] ='HOLLA'
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[116], line 2
      1 #Strings are immutable which means elements of a string cannot be changed on
ce they have been assigned
----> 2 str1[0:5] ='HOLLA'

TypeError: 'str' object does not support item assignment
```

In [117…

```python
del str1 # Delete a string
print(srt1)
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[117], line 2
      1 del str1 # Delete a string
----> 2 print(srt1)

NameError: name 'srt1' is not defined
```

String concatenation

In [118…

```python
# String concatenation
s1 = "Hello"
s2 = "Asif"
s3 = s1 + s2
print(s3)
```

```
HelloAsif
```

In [ ]: